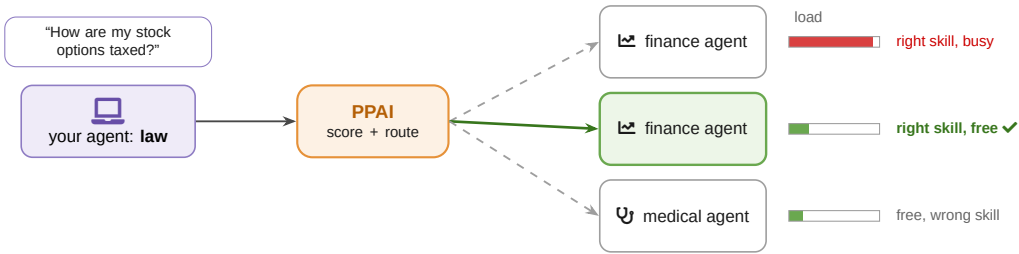
PPAI: Personalized LLM Agent Interoperability

for Collaborative Edge Intelligence

BitTorrent for expertise: send each question to a peer whose agent knows the answer, and is free.

Z. Wang, Q. Liu, K. Guo, H. Wang, J. Lin, Z. Hong, S. Guo · HKUST & SYSU · IEEE INFOCOM 2026



Group 21 · Adele Chinda (002831349) · Trilok Padhi (002716700)

CSC 8220 Advanced Computer Networks · Georgia State University · Instructor: Prof. Xiaojun Cao · Fall 2026

One INFOCOM paper, built from three older ideas

P2P lookup, LLM routing and congestion games meet in PPAI. Our Task III starts where PPAI stops.

THREAD 1 · 2002–2005

Peer-to-peer networks

BitTorrent, Kademlia, gossip: find data among churning peers.

THREAD 2 · 2024–2025

LLM routers

RouterDC, KABB, RouteLLM: best model per query, on *one* machine.

THREAD 3 · 2002

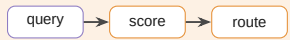
Congestion games

Selfish routing costs at most 5/3 of the optimum (Roughgarden & Tardos).

STATE OF THE ART · INFOCOM 2026

PPAI

Every user runs **one** specialised LLM agent at the edge. A query the local agent is weak at goes to a peer whose agent is strong at it **and** not overloaded.



reported: +7.96 pts, −16.34% time

TASK III · OUR PROJECT

Whom to trust?

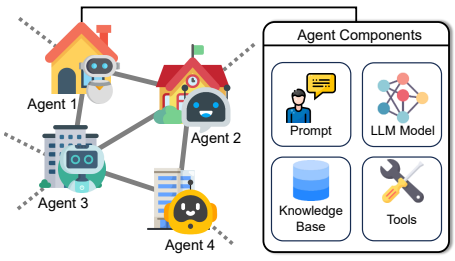
PPAI believes every peer's reported skill and every answer. What if a peer lies, or a group colludes?



PPAI = P2P lookup for *expertise* + LLM routing + a load-aware game between users.

Every laptop can now host a specialist agent, but only one

Small open models plus fine-tuning or RAG make personal agents cheap. Hosting every domain locally is not.



Paper Fig. 1: the vision. Each house is one user with one personalised agent.

What is an LLM agent?

An LLM plus a **system prompt**, a **knowledge base** and **tools**, which plans and acts toward a goal. Personalised by fine-tuning or by its own documents (RAG).

Why at the edge?

Privacy, low latency, no cloud bill: a consumer GPU runs a 7–8B model. The catch: it holds **one** specialist, not five.

The P2P analogy

2003 · BitTorrent

Missing a piece of a file? Download it from a peer who has it.

2026 · PPAI

Your agent is weak at a question? Send it to a peer whose agent is strong at it.

The twist: peers hold *different skills*, not copies of the same file, and a skill can be busy.

Each user hosts one expert; together, the network hosts all of them.

No agent wins everywhere, and the second-best is close

Paper §III: five 7–8B edge models (Qwen2.5, DeepSeek-R1-Distill, Nemotron-Code, HuatuoGPT-o1, Llama-3) on MMLU.

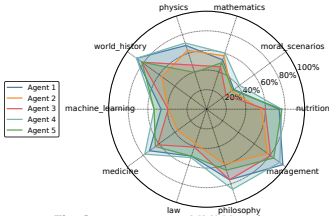


Fig. 2: accuracy per MMLU task



Fig. 3: how often each agent ranks top-1 / 2 / 3

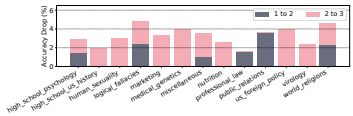


Fig. 4: points lost using the 2nd / 3rd best

FINDING 1

Agents are specialists. Agent 4 leads medicine, Agents 1–2 lead maths. Together they cover what none covers alone.

FINDING 2

A few agents win most tasks. Always routing to the top-1 piles queries onto Agents 1 and 4: congestion.

FINDING 3

Second-best is nearly as good. Top-1 $\rightarrow$ top-2 costs <5 pts on many tasks, and doubles the candidate pool.

Route to a *good-enough, free* peer, not always to the best one. That is the design in one line.

Related work, and the gap PPAI targets

LLM routers assume every model sits on one machine; P2P systems assume every peer holds the same kind of data.

	decentralised (P2P)	different skills per node	survives churn w/o retraining	load-aware routing
BitTorrent, Kademlia, gossip 2002–2005 · file sharing	●	○	●	◑
RouterDC NeurIPS 2024 · trained central router	○	●	○	○
KABB ICML 2025 · knowledge graph of experts	○	●	◑	○
RouteLLM and cost-aware routers ICLR 2025 · cost vs. quality	○	●	○	◑
PPAI INFOCOM 2026 · this paper	●	●	●	●

filled = designed for it · half = partly · empty = not addressed (as characterised in PPAI §II)

Orthogonal

Quantisation (GPTQ), distillation and pipeline parallelism make each agent cheaper. PPAI can use them unchanged.

Agents

Mind2Web, SWE-agent, MetaGPT; on-device: Mobile-Agent-v2, MiniRAG. All single-user.

PPAI claims to be the first to route queries to specialised agents across a churning P2P network.

Two challenges, two modules

Which peer *can* answer this query? And which peer *should*, when load changes faster than anyone can see?

CHALLENGE 1

Diverse agents that churn

Each node has different skills; agents join, leave and get updated. A trained central router treats agents as fixed classes, so every change means retraining.

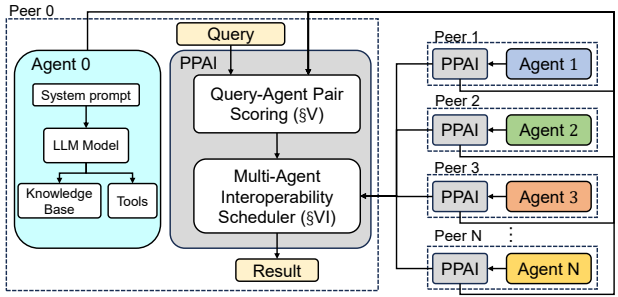
→ **§V: query–agent pair scoring**

CHALLENGE 2

Load you cannot see

A peer's load is stale by the time you hear it. Agents are interchangeable, so if everyone picks the locally best one, it overloads.

→ **§VI: Bayesian game scheduler**



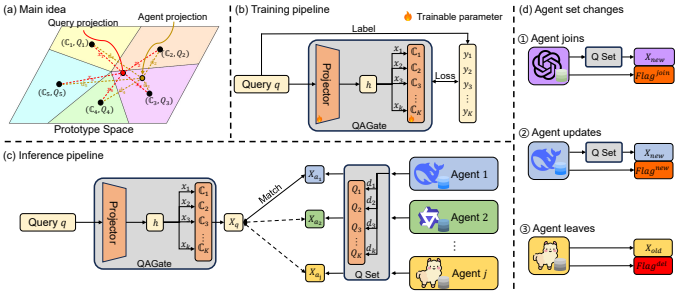
Paper Fig. 5: every peer runs the same PPAI module next to its own agent.

1. scoring → shortlist of capable agents **2. game → pick one, given load**

Scoring answers "who can?"; the game answers "who should, right now?"

Module 1: queries and agents share one prototype space

$K = 20$ learned prototypes act as fixed coordinates, so adding an agent never retrains the router.



Paper Fig. 6 (QAGate): (a) shared space, (b) training, (c) inference, (d) agent joins / updates / leaves.

① QUERY → RELEVANCE VECTOR

Sentence encoder + MLP gives $h(q)$; take its cosine to each prototype $\mathbb{C}_k$, sharpen (power α), keep the top-p, renormalise: $\tilde{f}(q) \in \mathbb{R}^K$. Trained with a KL loss against soft cluster labels.

② AGENT → CAPABILITY VECTOR

Test agent j on a small question set Q_k for each prototype: $p_j = [\mathrm{acc}_{j,1}, \ldots, \mathrm{acc}_{j,K}]$.

③ MATCH, AND HANDLE CHURN

Score $s(i,j) = \cos\left(\tilde{f}(q),\ p_j\right)$. Join, update or leave = gossip one K-vector with a flag and a timestamp.



An agent joining costs one broadcast of 20 numbers, not a retraining job.

Module 2: a Bayesian game decides who *should* serve

Each user trades relevance against an expected cost of delegation, using a belief about every peer's load.

User i's decision for query q_i

$$z_i = \arg\max_{j \in E_i} \underbrace{s(i,j)}_{\text{relevance}} - \beta \underbrace{c(i,j)}_{\text{cost of delegation}}$$

$$c(i,j) = \mathbb{E}_{\theta_j \sim b_i}[\rho_j] + t_j^{\text{infer}} + t_{ij}^{\text{trans}}$$

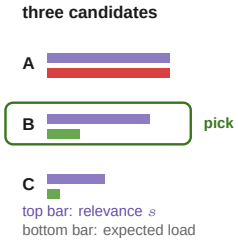
$$\mathbb{E}[\rho_j] = \min\{1,\ \rho_j + \delta(\lambda_j - \mu_j)\},\quad \rho_j = \lambda_j / \mu_j$$

Each peer has a private type $\theta_j = (\mu_j, \lambda_j)$ (service and arrival rate). User i keeps a belief $b_i(\theta_j)$ and updates it by Bayes' rule when j reports its state ω_j:

$$b_i^{\text{new}}(\theta_j) \propto P(\omega_j \mid \theta_j)\, b_i(\theta_j)$$

Algorithm 1, per query

1 score every agent: $s(i,j)$
2 shortlist E_i: agents with $s(i,j) \geq \theta_s$
3 ask each candidate for its load and timings
4 update beliefs (Bayes)
5 compute $c(i,j)$ and the utility
6 delegate to the arg max; serve locally if it is i



β is the one knob

small β: chase the most relevant agent (best accuracy, risk of queueing)
large β: spread queries over less loaded substitutes (lower latency, small accuracy loss)

The game turns Finding 3 into a rule: give up a little relevance to avoid a long queue.

What the paper proves about the game

Six results in §VI-D. The two that matter most: an equilibrium exists, and its inefficiency is bounded.

LEMMA 1

A best response always exists and is unique (finite candidate set, fixed tie-break).

LEMMA 2

Utility falls as expected load rises: $\partial U / \partial \, \mathbb{E}[\rho_j] = -\beta < 0.$

THEOREM 3

Beliefs converge almost surely to the true posterior (Doob's martingale theorem).

THEOREM 1

An equilibrium exists. With $\Phi(z) = \sum_i U(i, z_i)$ the game is an exact potential game, so a pure Bayesian Nash equilibrium exists; unique under strict concavity and consistent beliefs.

LEMMA 3

Users get there. Each best-response step strictly increases Φ, so the process stops at an equilibrium in finitely many steps.

THEOREM 2

Selfishness costs at most 5/3. For affine costs the Bayesian price of anarchy is $\leq \ 5/3$, via Roughgarden–Tardos for congestion games.



Claimed: selfish routing settles, and costs at most 5/3 of the optimum. We revisit this on slide 13.

Setup: five real 7–8B models, up to 1000 peers

Built on vLLM (inference) and libp2p (networking). Every number is the mean of 8 runs.

Five agents

A1	Qwen2.5-7B-Instruct	code, maths
A2	DeepSeek-R1-Distill-7B	reasoning
A3	Nemotron-Code-7B	code
A4	HuatuoGPT-o1-7B	medicine
A5	Llama-3-8B-Instruct	dialogue

Each also gets ShareGPT conversations as knowledge. Scaled to 50 / 100 / 500 / 1000 peers by copying the 5 models with different knowledge.

Workloads

MMLU, GSM8K (maths), MedQA (medical), ARC-C (science): 75% train QAGate, 25% become user queries.

AGIEval (Gaokao, GRE, SAT): held out as **out-of-distribution**.

Poisson arrivals at rate λ: $\lambda = 10$ in the main table, up to 100 queries/s in stress tests.

System

One Xeon Gold 6226 + RTX A6000 per agent.
Link inside a region: 5 ms, 2 Gbps.

QAGate: multilingual MPNet sentence encoder + 2-layer MLP; 20 prototype labels from agglomerative clustering.

Baselines and metrics

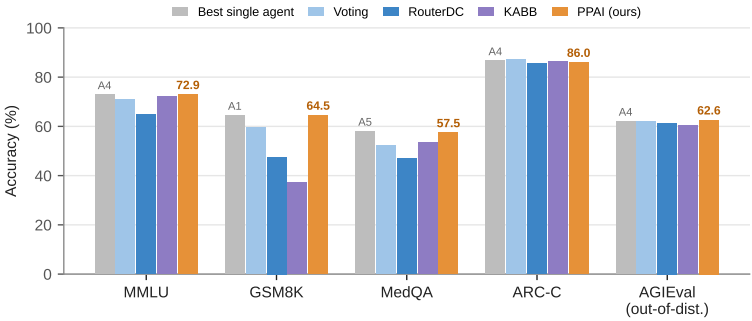
Single agent (A1–A5 alone) · **Majority voting** (ask all five, vote) · **RouterDC** (trained central router) · **KABB** (knowledge graph of experts). The centralised routers are re-implemented inside the same P2P system.

Metrics: average **accuracy** (routing quality) and end-to-end **process time** per query (queueing + inference).

Five specialists and one router per peer, compared with routers built for a single machine.

Result 1: as accurate as the right specialist

Paper Table I ($\lambda = 10$), redrawn. Grey = whichever single agent is best on that benchmark (A1, A4 or A5).



Data: paper Table I. Chart: ours. Full table in the backup slide.

+7.96

accuracy points on MMLU vs. RouterDC (72.94 vs. 64.98)

−16.3%

time per query on MMLU vs. Voting (21.05 vs. 25.16 ms)

Average over the 5 benchmarks: **68.7%**. The best *fixed* agent (A1) gets 65.8%; picking the best agent per benchmark gets 68.9%.

PPAI performs as if it knew in advance which specialist to use, without being told.

Result 2: spreading queries keeps latency flat

Stress tests on MMLU. Paper Figs. 8–10, cropped from the original PDF.

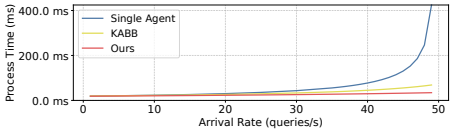


Fig. 8, arrival rate. A single agent's time explodes past ≈40 queries/s (queueing); PPAI stays lowest up to 50.

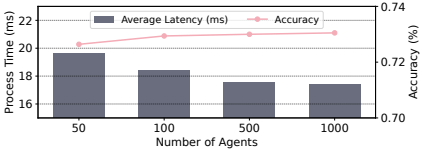
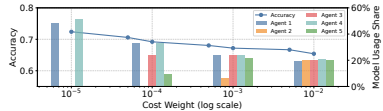
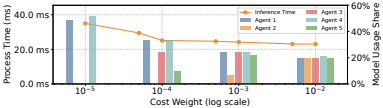


Fig. 10, scale ($\lambda = 100$). From 50 to 1000 peers, latency falls 19.6 → 17.4 ms; accuracy stays ≈73%.



(a) Accuracy.



(b) Process time.

Fig. 9, the β knob ($\lambda = 50$)

β from 10^{-5} to 10^{-2}:

accuracy ≈0.72 → 0.65

time ≈45 → 30 ms

Bars: usage moves from A1 and A4 to all five agents.

More peers and a larger β buy lower latency; accuracy pays only a little.

A closer look: what the evidence actually shows

Our own reading, for discussion. The system idea is strong; the evidence for load-awareness is the thinnest part.

1 · RESULTS

Headlines are single best-case comparisons

- "+7.96%" is MMLU vs. RouterDC; "−16.34%" is MMLU time vs. Voting.
- Against the best single agent on each benchmark, PPAI is +0.03, +0.01, −0.62, −0.68, +0.49 points.
- On GSM8K it takes 247 ms; RouterDC takes 103 ms.

The value is *not needing to know* the right agent, not beating it.

2 · THEORY

The proof removes the congestion it is about

- Theorem 1 uses $\Phi = \sum_i U(i, z_i)$. That is an exact potential only if user i's choice leaves every other user's utility unchanged.
- And indeed, load enters $U(i, j)$ only through i's *belief*, not through the others' choices: users never interact inside the model.
- Theorem 2 then borrows the 5/3 bound from congestion games, which needs exactly that interaction.

3 · SYSTEM

"Unobservable" load is polled on every query

- Algorithm 1 asks every candidate for $(\lambda_j, \mu_j, t^{\text{infer}}, t^{\text{trans}})$ on every query: extra round trips that are never measured.
- The load term is unitless, capped at 1, and added to latencies; β had to be swept over 10^{-5} to 10^{-2}.
- No experiment varies the churn rate or reports gossip and polling traffic.

A clean system idea; how it behaves with stale, shared load information is still open.

Conclusion

1 **Personal edge agents are specialists, so they should share.**

No agent wins everywhere, a few win most often, and the second-best is usually close (Findings 1–3).

2 **PPAI = prototype-anchored scoring + a Bayesian routing game.**

Queries and agents share a fixed 20-prototype space, so churn is one gossip message. Each user maximises relevance $-\ \beta\times$ cost of delegation, with an equilibrium and a price of anarchy $\leq 5/3$ (as claimed).

3 **It matches the best specialist per benchmark and stays fast under load.**

Up to +7.96 points vs. RouterDC and -16.34% time vs. voting on MMLU; flat latency up to 50 queries/s; scales to 1000 peers. Open: trust between peers (our Task III), stale load, real congestion between users.

Thank you. Next: our Task III progress.

Task III progress: whom should a peer trust?

PPAI believes every peer's self-reported skill and every answer. We break that assumption, then fix it.

THE GAP IN PPAI

Nothing checks the peers

A query goes to the peer that *claims* the best capability vector, and its answer is accepted as is. PPAI lists trust as future work.

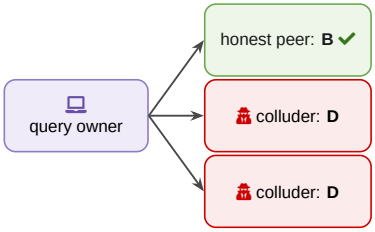
THREAT MODEL

Skill liars: claim 0.95 everywhere, answer badly

Colluders: all give the *same* wrong answer

Whitewashers: re-join under a new identity

Camouflage: answer correctly 30% of the time



a majority vote returns **D**: wrong

PPAI accuracy, 10% colluders

0.73 → 0.04

Dawid–Skene, 30% colluders

0.87 → 0.01

our simulator, PPAI's agent profiles

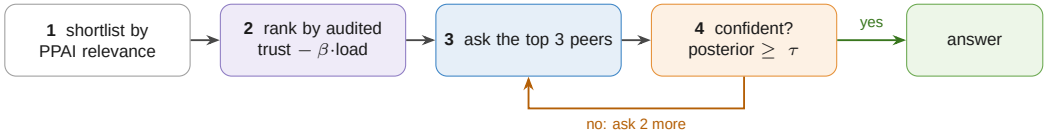
RESEARCH QUESTION

Among the peers that claim they can answer, **whom do we trust** and **how many do we ask**? There is no trusted third party, no pre-trusted peer, and every extra probe costs a round trip.

Trust-aware delegation is a networking problem: every verification is a probe that costs latency and GPU time.

Our method: TADA, Trust-Aware Delegation with Audits

Rank by audited trust, ask more peers only when unsure, and never let agreement outrun the audit record.



AUDIT-CAPPED TRUST

Two Beta reputations per peer and skill: *agreement* (cheap, but a coalition can fake it) and *gold-question audits* (cannot be faked).

$\hat{a}_{jk} = \min\{\text{agreement},\ \text{audit UCB}\}$

ANCHORED AGREEMENT

Learn from agreement only if the answer still wins when each peer is weighted by its *audit-only* accuracy.

Unaudited peers count as chance, so colluders cannot vouch for each other.

SPOT AUDITS (15% OF QUERIES)

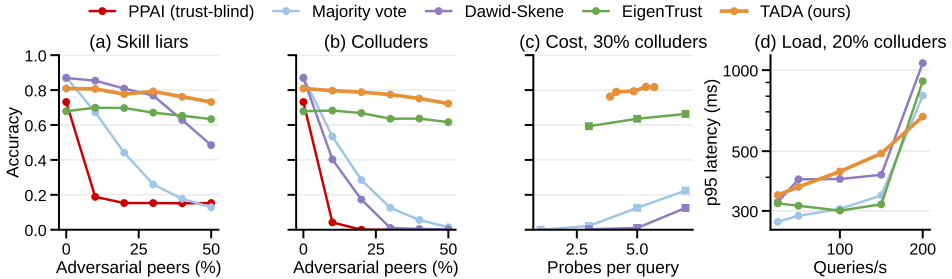
A gold question from PPAI's own per-prototype set Q_k goes to a peer we just relied on, or to the peer we know least about.

Newcomers and whitewashers are vetted on gold questions, not on user queries.

Reported skill is only a filter: trust is earned on gold questions and spread by anchored agreement.

Early results: robust to collusion, cheaper under load

Event-driven simulator: 50 peers with PPAI's five 7–8B agent profiles (Table I), FIFO queues, 5 seeds.



0.77 vs 0.64
accuracy with 30% colluders, vs EigenTrust (which is given 3 pre-trusted peers)

−17% probes
and the lowest p95 at 200 queries/s (672 vs 804–1062 ms)

The price
no attack: 6 pts below a 5-peer vote
sequential rounds: ≈+100 ms p95

Ablation: without audits, accuracy under collusion falls to 0.42; without anchoring, to 0.63.

Challenges, a first RL controller, and next steps

Done: simulator, four baselines, TADA and a Q-learning controller, with tests. Code: github.com/arrdel/ppai.

CHALLENGES, AND HOW WE FIXED THEM

- Wrong answers never agreed, so voting looked perfect → multiple choice with a shared distractor
- Agreement-only trust was captured by colluders (0.003) → audit cap and anchoring (+14 pts)
- The queue model left servers idle (spurious 12 s) → discrete-event simulator
- Up to 3 sequential rounds → ask 3 first (−25% p95)

PRELIMINARY RL (Q-LEARNING)

Learns *stop* vs. *ask 2 more*. State: confidence, round, load, skill. Reward: correct − probe cost − latency cost. Trained in simulation.

At 150 queries/s with 30% colluders:

p95 latency **576** → **310 ms**
probes per query **4.3** → **3.3**
accuracy −1.4 pts

Not yet better than the threshold rule at the high-accuracy end.

NEXT, TOWARD THE FINAL REPORT

- **Trilok:** real answers from the 5 models (vLLM); Azure and BurstGPT arrival traces
- **Adele:** better RL with per-round credit, an audit bandit and an adaptive audit rate
- **Trilok:** stronger attacks, such as spotting gold questions and Sybil swarms
- **Both:** IEEE final report ($\geq$6 pages) and the 25-min talk

Every number in these slides is regenerated by one script in the repository (task3/).

References

• Z. Wang, Q. Liu, K. Guo, H. Wang, J. Lin, Z. Hong, S. Guo. **PPAI: Enabling Personalized LLM Agent Interoperability for Collaborative Edge Intelligence.** IEEE INFOCOM 2026. doi:10.1109/INFOCOM59046.2026.11571683.

• B. Cohen. **Incentives build robustness in BitTorrent.** P2P Econ, 2003.

• P. Maymounkov, D. Mazières. **Kademlia: A peer-to-peer information system based on the XOR metric.** IPTPS, 2002.

• M. Jelasity, A. Montresor, O. Babaoglu. **Gossip-based aggregation in large dynamic networks.** ACM TOCS, 2005.

• S. Chen et al. **RouterDC: Query-based router by dual contrastive learning for assembling LLMs.** NeurIPS 2024.

• J. Zhang et al. **KABB: Knowledge-aware Bayesian bandits for dynamic expert coordination in multi-agent systems.** ICML 2025.

• I. Ong et al. **RouteLLM: Learning to route LLMs from preference data.** ICLR 2025.

• J. Snell, K. Swersky, R. Zemel. **Prototypical networks for few-shot learning.** NeurIPS 2017.

• T. Roughgarden, É. Tardos. **How bad is selfish routing?** Journal of the ACM, 2002.

• J. Li et al. **More agents is all you need.** TMLR 2024.

• A. P. Dawid, A. M. Skene. **Maximum likelihood estimation of observer error-rates using the EM algorithm.** J. Royal Stat. Soc. C, 1979.

• A. Jøsang, R. Ismail. **The Beta reputation system.** Bled eCommerce Conference, 2002.

• D. Oleson et al. **Programmatic gold: Targeted and scalable quality assurance in crowdsourcing.** HCOMP, 2011.

• S. Kamvar, M. Schlosser, H. Garcia-Molina. **The EigenTrust algorithm for reputation management in P2P networks.** WWW 2003.

• E. Friedman, P. Resnick. **The social cost of cheap pseudonyms.** J. Econ. & Mgmt. Strategy, 2001.

• C. Watkins, P. Dayan. **Q-learning.** Machine Learning, 1992.

Backup: Table I as printed in the paper

Accuracy (%) and process time (ms) per query, arrival rate $\lambda = 10$.

	MMLU		GSM8K		MedQA		ARC-C		AGIEval	
	Acc	Time	Acc	Time	Acc	Time	Acc	Time	Acc	Time
Single agent										
Agent 1	71.24	22.70	64.54	226.80	46.94	56.00	85.32	15.47	61.05	52.44
Agent 2	54.43	22.70	20.91	290.63	19.62	50.15	17.75	15.47	12.75	53.85
Agent 3	63.02	25.16	33.33	290.63	37.21	50.83	68.94	17.65	48.38	68.07
Agent 4	72.91	22.70	33.64	275.94	50.86	50.38	86.69	15.88	62.13	53.85
Agent 5	64.01	22.70	59.39	306.50	58.08	49.70	74.74	15.88	39.18	53.61
Multi-agents										
Voting	70.87	25.16	59.39	306.50	52.12	50.83	87.03	17.65	62.06	68.07
RouteDC [13]	64.98	19.56	47.58	103.24	46.94	49.48	85.67	15.13	61.36	43.81
KABB [14]	72.29	22.40	37.27	287.25	53.53	42.67	86.35	15.96	60.20	49.64
Ours	72.94	21.05	64.55	247.31	57.46	50.37	86.01	14.69	62.62	42.12

PPAI is best or within about 1 point of the best on every benchmark, and never the slowest multi-agent method.